

۱. چون باعث میشه هر بخش فقط یک مسؤلیت داشته باشه و در آینده توسعه بدون تغییرات گسترده و سختی کار پیش بره .

مثال :

```
namespace HomeWork4.Task1
{
    0 references | 0 changes | 0 authors, 0 changes
    public class Book
    {
        1 reference | 0 changes | 0 authors, 0 changes
        public string Title { get; set; }

        0 references | 0 changes | 0 authors, 0 changes
        public void PrintBook()
        {
            Console.WriteLine(Title);
        }

        0 references | 0 changes | 0 authors, 0 changes
        public void SaveBook()
        {
            Console.WriteLine("Book saved");
        }
    }
}
```

در اینجا کلاس Book دوتا مسؤلیت دارد:

۱. ذخیره اطلاعات کتاب

۲. نمایش اطلاعات کتاب

راه حل :

هر کلاس یک کار متفاوت رو انجام بده :

سه کلاس متفاوت میسازیم که کارهای مختلفی که داخل کلاس قبلی

داشتیم رو جداگانه انجام بده

```
BookRepository.cs | BookPrinter.cs | Books.cs | Program.cs
HomeWork4
1 using System;
2 using System.Collections.Generic;
3 using System.Text;
4
5 namespace HomeWork4.Task1
6 {
7     0 references | 0 changes | 0 authors, 0 changes
8     public class BookRepository
9     {
10         0 references | 0 changes | 0 authors, 0 changes
11         public void Save(Book book)
12         {
13             Console.WriteLine("Book Saved");
14         }
15     }
16 }
```

```
BookRepository.cs | BookPrinter.cs | Books.cs | Program.cs
HomeWork4
1 using System;
2 using System.Collections.Generic;
3 using System.Text;
4
5 namespace HomeWork4.Task1
6 {
7     2 references | 0 changes | 0 authors, 0 changes
8     public class Book
9     {
10         1 reference | 0 changes | 0 authors, 0 changes
11         public string Title { get; set; }
12     }
13 }
```

```
BookRepository.cs | BookPrinter.cs | Books.cs | Program.cs
HomeWork4
1 using System;
2 using System.Collections.Generic;
3 using System.Text;
4
5 namespace HomeWork4.Task1
6 {
7     0 references | 0 changes | 0 authors, 0 changes
8     public class BookPrinter
9     {
10         0 references | 0 changes | 0 authors, 0 changes
11         public void Print(Book book)
12         {
13             Console.WriteLine(book.Title);
14         }
15     }
16 }
```

۲. اصل OCP میگوید کلاس ها باید برای گسترش باز و برای تغییر بسته باشند . این یعنی افزودن ویژگی های جدید بدون تغییر در کد موجود .

در کد نشان داده شده اگر ما بخواهیم حیوان جدیدی اضافه بکنیم باید کد را تغییر بدیم .

```
Animal.cs* Program.cs
HomeWork4 HomeWork4.Task2.Anim
1 using System;
2 using System.Collections.Generic;
3 using System.Text;
4
5 namespace HomeWork4.Task2
6 {
7     0 references 0 changes 0 authors 0 changes
8     public class Animal
9     {
10         1 references 0 changes 0 authors 0 changes
11         public string Type { get; set; }
12
13         0 references 0 changes 0 authors 0 changes
14         public void MakeSound()
15         {
16             if (Type == "Cat")
17             {
18                 Console.WriteLine("Meow");
19             }
20             else if (Type == "Dog")
21             {
22                 Console.WriteLine("Woof");
23             }
24         }
25     }
26 }
```

راه حل :

یک کلاس پایه میسازیم و اگر بخواهیم شی جدید اضافه کنیم دیگه کلاس پایه رو تغییر نمیدیم و کلاس های دیگه میسازیم .

```
Animal.cs* Program.cs
HomeWork4 HomeWork4.Task2
1 using System;
2 using System.Collections.Generic;
3 using System.Text;
4
5 namespace HomeWork4.Task2
6 {
7     3 references 0 changes 0 authors 0 changes
8     public class Animal
9     {
10         3 references 0 changes 0 authors 0 changes
11         public virtual void MakeSound()
12         {
13         }
14     }
15
16     0 references 0 changes 0 authors 0 changes
17     public class Cat : Animal
18     {
19         1 reference 0 changes 0 authors 0 changes
20         public override void MakeSound()
21         {
22             Console.WriteLine("Meow");
23         }
24     }
25
26     0 references 0 changes 0 authors 0 changes
27     public class Dog : Animal
28     {
29         1 reference 0 changes 0 authors 0 changes
30         public override void MakeSound()
31         {
32             Console.WriteLine("Woof");
33         }
34     }
35
36     0 references 0 changes 0 authors 0 changes
37     public class Cow : Animal
38     {
39         1 reference 0 changes 0 authors 0 changes
40         public override void MakeSound()
41         {
42             Console.WriteLine("Moo");
43         }
44     }
45 }
```

۳. اصل LSP بیان می کند که کلاس های فرزند باید بتوانند بدون تغییر رفتار جایگزین کلاس های والد شوند .
چون اگر کلاس فرزند نتواند بدون مشکل جایگزین کلاس والد شود، برنامه ممکن است خطا بدهد و عملکرد کلی رو مختل بکند .

مثال : در این جا مشکل این است که مربع از نظر رفتار جایگزین مناسبی برای مستطیل نیست . در مستطیل می توان عرض و ارتفاع را مستقل از هم تغییر داد، اما در مربع همیشه باید برابر باشند.

```

square.cs* x Rectangle.cs* Program.cs
HomeWork4
1 using System;
2 using System.Collections.Generic;
3 using System.Text;
4
5 namespace HomeWork4.Task3
6 {
7     0 references | 0 changes | 0 authors, 0 changes
8     public class Square : Rectangle
9     {
10         4 references | 0 changes | 0 authors, 0 changes
11         public override int Width
12         {
13             get
14             {
15                 base.Width = value;
16                 base.Height = value;
17             }
18         }
19         4 references | 0 changes | 0 authors, 0 changes
20         public override int Height
21         {
22             get
23             {
24                 base.Width = value;
25                 base.Height = value;
26             }
27         }
28     }
29 }

```

```

square.cs* x Rectangle.cs* Program.cs
HomeWork4
1 using System;
2 using System.Collections.Generic;
3 using System.Text;
4
5 namespace HomeWork4.Task3
6 {
7     1 reference | 0 changes | 0 authors, 0 changes
8     public class Rectangle
9     {
10         4 references | 0 changes | 0 authors, 0 changes
11         public virtual int Width { get; set; }
12         4 references | 0 changes | 0 authors, 0 changes
13         public virtual int Height { get; set; }
14
15         0 references | 0 changes | 0 authors, 0 changes
16         public int GetArea()
17         {
18             return Width * Height;
19         }
20     }
21 }

```

راه حل :

یک کلاس والد بنویسیم که هر دو ازش ارث بری کنند .

```

Shape.cs* x square.cs* Rectangle.cs* Program.cs
HomeWork4
1 using System;
2 using System.Collections.Generic;
3 using System.Text;
4
5 namespace HomeWork4.Task3
6 {
7     1 reference | 0 changes | 0 authors, 0 changes
8     public abstract class Shape
9     {
10         1 reference | 0 changes | 0 authors, 0 changes
11         public abstract int GetArea();
12     }
13 }

```

```

Shape.cs* square.cs* Rectangle.cs* Program.cs
HomeWork4
1 using System;
2 using System.Collections.Generic;
3 using System.Text;
4
5 namespace HomeWork4.Task3
6 {
7     0 references | 0 changes | 0 authors, 0 changes
8     public class Rectangle
9     {
10         1 reference | 0 changes | 0 authors, 0 changes
11         public virtual int Width { get; set; }
12         1 reference | 0 changes | 0 authors, 0 changes
13         public virtual int Height { get; set; }
14
15         0 references | 0 changes | 0 authors, 0 changes
16         public int GetArea()
17         {
18             return Width * Height;
19         }
20     }
21 }

```

```

Shape.cs* square.cs* Rectangle.cs* Program.cs
HomeWork4
1 using System;
2 using System.Collections.Generic;
3 using System.Text;
4
5 namespace HomeWork4.Task3
6 {
7     0 references | 0 changes | 0 authors, 0 changes
8     public class Square : Shape
9     {
10         2 references | 0 changes | 0 authors, 0 changes
11         public int Side { get; set; }
12
13         1 reference | 0 changes | 0 authors, 0 changes
14         public override int GetArea()
15         {
16             return Side * Side;
17         }
18     }
19 }

```

۴. چون اگر یک کلاس مجبور شود متدهایی را پیاده‌سازی کند که به آن‌ها نیازی ندارد کد پیچیده تر می‌شود و ممکن است متدهای خالی یا بدون استفاده ایجاد شده و توسعه برنامه سخت شود

مثال :

```
Robot.cs* Human.cs* IWorker.cs* Program.cs
HomeWork4
1 using System;
2 using System.Collections.Generic;
3 using System.Text;
4
5 namespace HomeWork4.Task4
6 {
7     2 references | 0 changes | 0 authors, 0 changes
8     public interface IWorker
9     {
10         2 references | 0 changes | 0 authors, 0 changes
11         void Work();
12         2 references | 0 changes | 0 authors, 0 changes
13         void Eat();
14     }
15 }
```

در اینجا ربات نیازی به غذا خوردن ندارد ولی چون داخل اینترفیس متد Eat() وجود دارد باید اون رو پیاده سازی کرد. حتی اگر داخلش

از exception استفاده بشه

```
Robot.cs* Human.cs* IWorker.cs* Program.cs
HomeWork4
1 using System;
2 using System.Collections.Generic;
3 using System.Text;
4
5 namespace HomeWork4.Task4
6 {
7     0 references | 0 changes | 0 authors, 0 changes
8     public class Robot : IWorker
9     {
10         1 reference | 0 changes | 0 authors, 0 changes
11         public void Work()
12         {
13             Console.WriteLine("Robot is working.");
14         }
15         1 reference | 0 changes | 0 authors, 0 changes
16         public void Eat()
17         {
18             throw new NotImplementedException();
19         }
20     }
```

```
Robot.cs* Human.cs* IWorker.cs* Program.cs
HomeWork4
1 using System;
2 using System.Collections.Generic;
3 using System.Text;
4
5 namespace HomeWork4.Task4
6 {
7     0 references | 0 changes | 0 authors, 0 changes
8     public class Human : IWorker
9     {
10         1 reference | 0 changes | 0 authors, 0 changes
11         public void Work()
12         {
13             Console.WriteLine("Human is working.");
14         }
15         1 reference | 0 changes | 0 authors, 0 changes
16         public void Eat()
17         {
18             Console.WriteLine("Human is eating.");
19         }
20     }
```


راه حل : اینترفیسمون رو کوچک تر میکنیم و یک اینترفیس دیگه فقط برای خوردن میسازیم.

```

// IEat.cs
using System;
using System.Collections.Generic;
using System.Text;

namespace HomeWork4.Task4
{
    public interface IEat
    {
        void Eat();
    }
}

// Robot.cs
using System;
using System.Collections.Generic;
using System.Text;

namespace HomeWork4.Task4
{
    public class Robot : IWork
    {
        public void Work()
        {
            Console.WriteLine("Robot is working.");
        }
    }
}

// Human.cs
using System;
using System.Collections.Generic;
using System.Text;

namespace HomeWork4.Task4
{
    public class Human : IWork, IEat
    {
        public void Work()
        {
            Console.WriteLine("Human is working.");
        }

        public void Eat()
        {
            Console.WriteLine("Human is eating.");
        }
    }
}

// IWork.cs
using System;
using System.Collections.Generic;
using System.Text;

namespace HomeWork4.Task4
{
    public interface IWork
    {
        void Work();
    }
}
    
```

۵. این اصل میگوید که مازول های سطح بالا به مازول های سطح پایین نباید دسترسی داشته باشند بلکه هردو باید به interface/ abstract کلاس ها وابسته باشند . با انجام این کار می توان بدون تغییر کلاس اصلی، کلاس های جدید را جایگزین یا اضافه کرد .

مثال : در اینجا کلاس Switch مستقیماً به LightBulb وابسته است

اگر بعداً بخواهیم به جای لامپ، پنکه را روشن کنیم

باید کلاس Switch را تغییر دهیم

```

// Switch.cs
using System;
using System.Collections.Generic;
using System.Text;

namespace HomeWork4.Task5
{
    public class Switch
    {
        private LightBulb bulb = new LightBulb();

        public void Press()
        {
            bulb.TurnOn();
        }
    }
}

// LightBulb.cs
using System;
using System.Collections.Generic;
using System.Text;

namespace HomeWork4.Task5
{
    public class LightBulb
    {
        public void TurnOn()
        {
            Console.WriteLine("Light is ON");
        }
    }
}
    
```

راه حل : یک اینترفیس مانند ایجاد می‌کنیم و کلاس‌های آن را پیاده‌سازی می‌کنند. سپس کلاس به جای وابستگی به کلاس دیگر ، به آن اینترفیس وابسته می‌شود این کار باعث کاهش وابستگی برنامه می‌شود.

```

// ISwitchable.cs
using System;
using System.Collections.Generic;
using System.Text;

namespace HomeWork4.Task5
{
    public interface ISwitchable
    {
        void TurnOn();
    }
}

// Switch.cs
using HomeWork4.Task5.Interfaces;
using System;
using System.Collections.Generic;
using System.Text;

namespace HomeWork4.Task5
{
    public class Switch : ISwitchable
    {
        private ISwitchable device;

        public Switch(ISwitchable device)
        {
            this.device = device;
        }

        public void Press()
        {
            device.TurnOn();
        }
    }
}

// Fan.cs
using HomeWork4.Task5.Interfaces;
using System;
using System.Collections.Generic;
using System.Text;

namespace HomeWork4.Task5
{
    public class Fan : ISwitchable
    {
        public void TurnOn()
        {
            Console.WriteLine("Fan is ON");
        }
    }
}

// IEat.cs
using System;
using System.Collections.Generic;
using System.Text;

namespace HomeWork4.Task4.Interfaces
{
    public interface IEat
    {
        void Eat();
    }
}

// LightBulb.cs
using HomeWork4.Task5.Interfaces;
using System;
using System.Collections.Generic;
using System.Text;

namespace HomeWork4.Task5
{
    public class LightBulb : ISwitchable
    {
        public void TurnOn()
        {
            Console.WriteLine("Light is ON");
        }
    }
}

// Program.cs
using System;
using System.Collections.Generic;
using System.Text;

namespace HomeWork4.Task5.Interfaces
{
    public interface ISwitchable
    {
        void TurnOn();
    }
}

```

۶. گزینه B تغییر این اصل را نقض میکند و برای توسعه و گسترش باید کلاس‌های دیگه ای بسازیم و از ارث بری استفاده نکنیم